HIREFLOW

AI-Powered B2B Hiring Process Orchestration SaaS

Software Design · Academic Project Report · April 2026

Team Member	Enrolment No.
Priyansh Satija	2401010355
Tathagat Harsh	2401010477
Bhavay Goyal	2401010126
Pranav Sehgal	2401010332
Preet	2401010352

Newton School of Technology · Rishihood University

https://b2-b-interview-hiring-process-orche.vercel.app/

01 PROJECT OVERVIEW

HireFlow is a production-grade, AI-assisted B2B SaaS platform built to automate and streamline end-to-end recruitment workflows. The system uses the Groq API (Llama 3 via Groq's ultra-fast inference engine) for intelligent resume parsing and candidate evaluation, paired with a Clean Architecture backend and a real-time Next.js frontend.

Live deployment: https://b2-b-interview-hiring-process-orche.vercel.app/

Problem Statement

Traditional hiring workflows are slow, inconsistent, and heavily manual — recruiters spend hours parsing resumes and shuffling spreadsheets with no real-time pipeline visibility. HireFlow eliminates this by automating repetitive tasks and centralising hiring intelligence into one multi-tenant platform.

Key Features

- **Intelligent Resume Parsing** — Groq API (Llama 3) extracts structured data from PDF, DOCX, and TXT files at ultra-low latency
- **AI Candidate Evaluation** — LLM-driven job-fit analysis and structured scoring via Groq inference
- **Multi-Stage Kanban Pipeline** — Visual drag-and-drop board with live WebSocket collaboration
- **Smart Scoring System** — Pluggable strategy-based candidate ranking (skills, experience, education, culture fit)
- **Analytics Dashboard** — Real-time hiring funnel metrics and team performance insights
- **Multi-Tenant Architecture** — Secure org-level data isolation with tenantId scoped to every query
- **Role-Based Access Control** — Admin · Recruiter · Hiring Manager · Interviewer — fine-grained permissions

02 TECHNOLOGY STACK

Layer	Technologies
Backend	Node.js · TypeScript · Express.js · Prisma ORM · PostgreSQL · Redis · Bull Queues · JWT
Frontend	Next.js 14 · React 18 · TypeScript · Tailwind CSS · Recharts · React DnD Kit
AI / ML	Groq API · Llama 3 (llama3-8b-8192) · NLP-based skill extraction · structured LLM prompting
Database	PostgreSQL 15+ · Prisma Migrations · Supabase (optional hosted backend)
Real-time	WebSockets (Socket.io) — live pipeline updates, candidate stage changes, notifications
DevOps	Docker · Vercel (frontend) · environment-based configuration management
Testing	Jest · integration tests · target 80 %+ coverage

03 SYSTEM ARCHITECTURE

HireFlow follows Clean Architecture with strict four-layer separation of concerns. Business logic is entirely independent of frameworks, databases, and external AI services.

Architecture Layers

- **Presentation Layer** — Express REST API + Next.js UI. Handles HTTP requests, serialisation, and rendering — no business logic
- **Application Layer** — Use Cases + Orchestrators. Co-ordinates domain operations without coupling to infrastructure
- **Domain Layer** — Entities · Interfaces · Value Objects. Pure business rules with zero external dependencies
- **Infrastructure Layer** — Prisma ORM · Groq AI Service · File Storage · Redis Queue. All I/O adapters live here

Dependency Rule

Inner layers never depend on outer layers. Interfaces flow inward — the domain layer has no import from Express, Prisma, or Groq. This means the entire business logic can be unit-tested without spinning up any infrastructure.

Multi-Tenant Design

Each tenant (organisation) has isolated data. Every Prisma query is scoped by tenantId at the repository level, ensuring zero cross-tenant leakage. RBAC sits on top, with four distinct role capability sets enforced in middleware before any controller logic runs.

AI Integration (Groq + Llama 3)

The GroqAIService implements the IAIService interface, which exposes parse() and analyze() methods. The application layer depends only on IAIService — making the AI provider fully swappable. At runtime, GroqAIService sends structured prompts to the Llama 3 model via Groq's REST API, receiving JSON-formatted skill extraction and job-fit scores in under 500 ms.

04 DESIGN PATTERNS IMPLEMENTED

HireFlow implements six software design patterns, each applied where it yields the highest architectural return. The table below gives a quick overview; detailed explanations follow.

Pattern	Location in Codebase	Architectural Justification
Repository	domain/repositories/ IRepository.ts · infrastructure/repositories/	Decouples business logic from the database; Prisma can be replaced without touching any use case
Strategy	domain/strategies/ IScoringStrategy.ts · application/services/ScoringService.ts	Interchangeable scoring algorithms at runtime; add new strategies without modifying the scorer
Factory	infrastructure/parsers/ ResumeParserFactory.ts	Centralises file-type based parser creation; new formats plugged in without touching callers
Observer	domain/events/ · infrastructure/observers/	Event-driven WebSocket notifications for CandidateCreated, EvaluationCompleted, StageChanged
Singleton	infrastructure/di/ · infrastructure/config/	Shared DI Container, Config Manager, Event Emitter — consistent global state in-process
Dependency Injection	All service constructors across application/ and infrastructure/	Eliminates hard-coded dependencies; full testability via constructor injection of interfaces

Repository Pattern

IRepository<T> defines findById, findAll, save, update, and delete. PrismaCandidateRepository and PrismaJobRepository implement this contract in the infrastructure layer. The application layer only ever holds a reference to the interface — swapping PostgreSQL for any other database requires zero changes outside infrastructure.

Strategy Pattern

IScoringStrategy exposes calculate(candidate, job), getName(), and getWeight(). Concrete strategies — SkillsMatchStrategy, ExperienceStrategy, EducationStrategy — are registered with ScoringService and their weighted outputs are aggregated at runtime. New scoring dimensions can be added without modifying existing strategies, directly satisfying the Open/Closed Principle.

Factory Pattern

ResumeParserFactory inspects the uploaded file extension and returns the appropriate IResumeParser implementation (PDFResumeParser, DOCXResumeParser, TXTResumeParser). All calling code works only with the interface — the factory decision is entirely hidden.

Observer Pattern

Domain events are emitted through a centralised EventEmitter singleton. Infrastructure observers subscribe and react: dispatching WebSocket messages to connected recruiters, triggering Bull queue jobs for async processing, and writing structured audit log entries. Core use cases emit events but have no knowledge of how they are consumed.

05 SOLID PRINCIPLES

S — Single Responsibility

Every class has exactly one reason to change. ResumeParsingService handles only parsing. GroqAIService handles only Groq communication. AuditService handles only audit writes. No class mixes infrastructure and domain concerns.

O — Open / Closed

The Strategy pattern for scoring and the Factory pattern for resume parsing make the system open for extension and closed for modification. Adding a new scoring dimension or a new file format requires writing one new class — no existing code is touched.

L — Liskov Substitution

All IRepository implementations are fully substitutable. Any concrete repository can replace its interface at the DI wiring point without breaking the application layer. All IScoringStrategy implementations are interchangeable in ScoringService without altering its logic.

I — Interface Segregation

Interfaces are narrow: IRepository<T>, IScoringStrategy, IResumeParser, IAIService, IQueueObserver. No class implements methods it does not use. Controllers depend only on the exact interface methods they call — there are no fat interfaces.

D — Dependency Inversion

High-level application services depend on abstract interfaces, not concrete implementations. The DI Container wires all concrete classes at startup. This is the architectural spine of the system — it is what allows every service to be tested in complete isolation.

06 ADDITIONAL DESIGN PRINCIPLES

- **DRY** — Shared utilities, base repository logic, and common validators are reused. No duplicate query or validation logic exists across the codebase
- **KISS** — Each module has a clear, single purpose. Complex operations are composed from well-named, small units — no clever abstractions for their own sake
- **YAGNI** — Only features explicitly required by the hiring workflow were built. No speculative generalisations or unused extension points
- **Separation of Concerns** — Presentation, business logic, and data access are strictly separated across the four architectural layers
- **High Cohesion · Loose Coupling** — Each module is internally cohesive around one domain concept and externally coupled only through typed interfaces

07 API ENDPOINTS

The backend exposes 20+ RESTful endpoints organised by domain resource. Full interactive documentation is available at /api-docs on the running server.

Method	Endpoint	Description
POST	/api/v1/candidates/upload	Upload resume — triggers Groq AI parse + scoring pipeline
GET	/api/v1/candidates	List all candidates scoped to authenticated tenant
POST	/api/v1/evaluations	Trigger AI evaluation of a candidate against a job description
GET	/api/v1/pipelines/:id/board	Retrieve full Kanban board state for a hiring pipeline
GET	/api/v1/analytics/dashboard	Real-time hiring funnel metrics and team performance stats
POST	/api/v1/auth/login	JWT authentication — returns signed token with role claims
PATCH	/api/v1/candidates/:id/stage	Move candidate to a new pipeline stage (triggers Observer event)

Real-time WebSocket Events

- **candidate:created** — Broadcasts to all active recruiters when a new candidate is added
- **evaluation:completed** — Notifies when Groq AI scoring finishes for a candidate
- **stage:changed** — Live Kanban board update when a candidate moves to a new pipeline stage

08 UML DIAGRAMS

8.1 Use Case Diagram

Two primary actors interact with HireFlow: the Recruiter and the Admin. The Recruiter can Upload Resume, Move Candidate Stage, and View Dashboard. The Admin additionally accesses the Audit Log Viewer. Upload Resume includes AI Parse Resume, which in turn includes Intelligent Scoring — reflecting the automated Groq AI processing chain triggered on every submission. The External AI Service actor represents the Groq API.

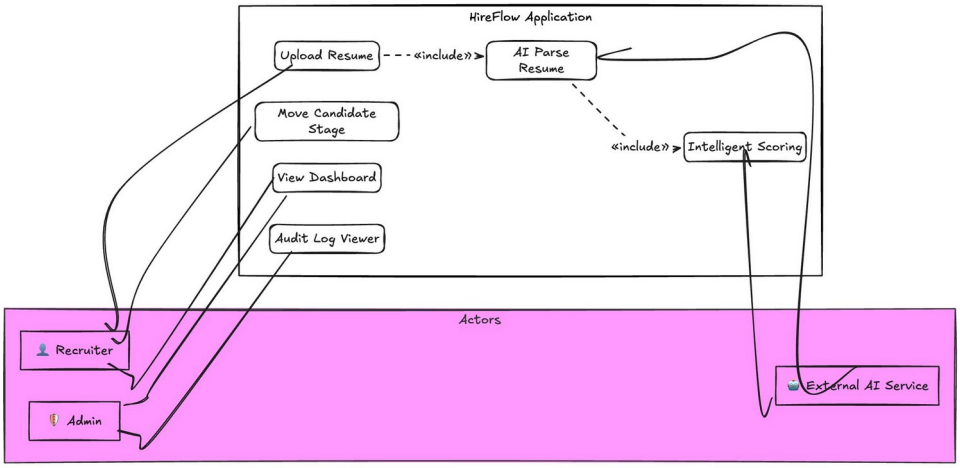

Figure 1 — HireFlow Use Case Diagram: Recruiter, Admin, and External AI Service (Groq) actors

8.2 Class Diagram

CandidateController exposes uploadResume() and getCandidate(), delegating to ProcessResumeUseCase. The use case coordinates GroqAIService (which implements IAIService via Llama3_Integration), ICandidateRepository (implemented by PrismaCandidateRepository), LocalFileStorage, and CandidateScorer. The Candidate entity holds id (UUID) and email. This structure maps directly to the Clean Architecture layers: controllers in presentation, use cases in application, interfaces in domain, and concrete classes in infrastructure.

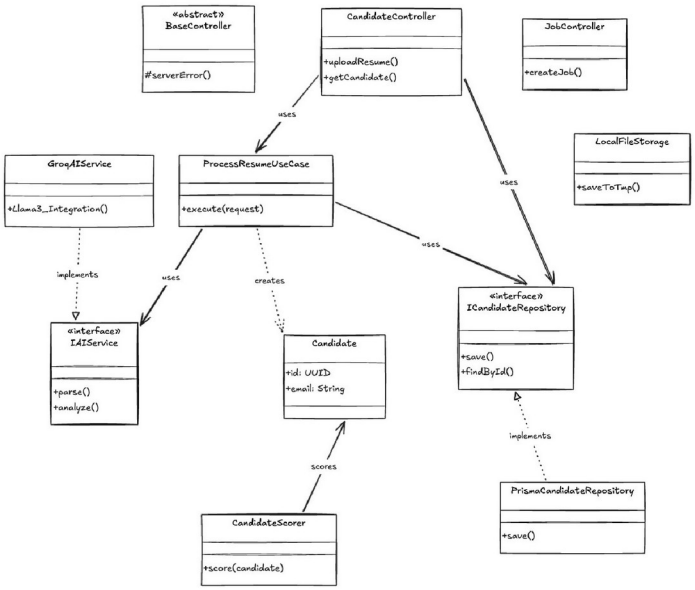

Figure 2 — HireFlow Class Diagram: Controllers, Use Cases, Groq AI Service, Repository, and Entities

8.3 Entity-Relationship Diagram

A TENANT owns many USERs and hosts many JOBs, and also manages many CANDIDATEs. Each JOB uses a PIPELINE_TEMPLATE, which contains multiple PIPELINE_STAGEs. A CANDIDATE is associated with a current PIPELINE_STAGE and is scored_by one or more EVALUATIONs. The CANDIDATE entity carries id, email, resumeUrl, and tenantId (FK) — enforcing multi-tenant isolation at the schema level.

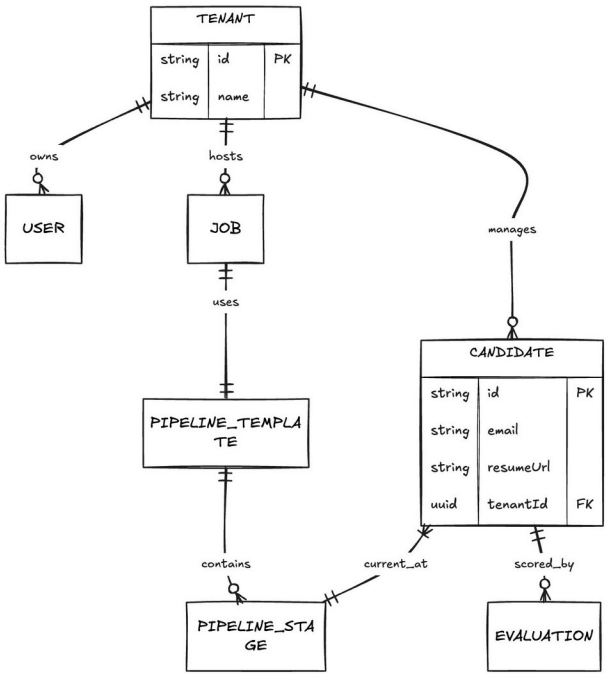

Figure 3 — HireFlow ER Diagram: Multi-tenant data model with Tenant, Job, Candidate, Pipeline, and Evaluation

09 PROJECT ACHIEVEMENTS

✦ 10 + database tables with full relational integrity	✦ 20 + REST API endpoints across all domain resources
✦ 6 design patterns with clear architectural justification	✦ Clean Architecture — strict 4-layer separation enforced
✦ Groq AI (Llama 3) integration for <500 ms resume processing	✦ Multi-tenant SaaS with complete data isolation per organisation
✦ RBAC across 4 distinct user roles	✦ Real-time WebSocket pipeline with responsive UI and dark mode
✦ Full test suite — 80 %+ coverage target	✦ Fully deployed on Vercel · live and accessible

10 CONCLUSION

HireFlow demonstrates how modern software engineering principles — Clean Architecture, SOLID, and six design patterns — combine with Groq AI to solve a real B2B hiring problem at scale. The Groq API's ultra-fast Llama 3 inference brings genuine AI value to the platform: sub-second resume parsing and job-fit evaluation that would be impractical with a slower model. Every architectural decision was made with long-term maintainability in mind, producing a system that is extensible, fully multi-tenant, production-deployed, and built to grow.

Live Application: https://b2-b-interview-hiring-process-orche.vercel.app/
Repository: github.com/couchpotato027/B2B-Interview-Hiring-Process-Orchestration-SaaS